composer/semver

Semver (Semantic Versioning) library that offers utilities, version constraint parsing and validation.

Originally written as part of composer/composer, now extracted and made available as a stand-alone library.

Installation

Install the latest version with:

```
composer require composer/semver
```

Requirements

- PHP 5.3.2 is required but using the latest version of PHP is highly recommended.

Version Comparison

For details on how versions are compared, refer to the Versions article in the documentation section of the getcomposer.org website.

Basic usage

Comparator

The `Composer\Semver\Comparator` class provides the following methods for comparing versions:

- greaterThan($v1, $v2)
- greaterThanOrEqualTo($v1, $v2)
- lessThan($v1, $v2)
- lessThanOrEqualTo($v1, $v2)
- equalTo($v1, $v2)
- notEqualTo($v1, $v2)

Each function takes two version strings as arguments and returns a boolean. For example:

```
use Composer\Semver\Comparator;

Comparator::greaterThan('1.25.0', '1.24.0'); // 1.25.0 > 1.24.0
```

Semver

The `Composer\Semver\Semver` class provides the following methods:

- satisfies($version, $constraints)
- satisfiedBy(array $versions, $constraint)
- sort($versions)
- rsort($versions)

Intervals

The `Composer\Semver\Intervals` static class provides a few utilities to work with complex constraints or read version intervals from a constraint:

```
use Composer\Semver\Intervals;

// Checks whether $candidate is a subset of $constraint
Intervals::isSubsetOf(ConstraintInterface $candidate, ConstraintInterface $constraint);

// Checks whether $a and $b have any intersection, equivalent to $a->matches($b)
Intervals::haveIntersections(ConstraintInterface $a, ConstraintInterface $b);

// Optimizes a complex multi constraint by merging all intervals down to the smallest
// possible multi constraint. The drawbacks are this is not very fast, and the resulting
// multi constraint will have no human readable prettyConstraint configured on it
Intervals::compactConstraint(ConstraintInterface $constraint);

// Creates an array of numeric intervals and branch constraints representing a given constr
Intervals::get(ConstraintInterface $constraint);

// Clears the memoization cache when you are done processing constraints
Intervals::clear()
```

See the class docblocks for more details.

License

composer/semver is licensed under the MIT License, see the LICENSE file for details.